No-Code Form Builder & Workflow

Sprint 1: SRS

1. Introduction

1.1 Purpose

This document details the functional and non-functional requirements for the No-Code Form Builder Application, a core component of DISHA (a single tool for comprehensive organizational management). The primary goal of this tool is to empower non-technical users to build forms effortlessly and effectively. The form builder is capable of creating a wide range of forms, from simple feedback and application forms to complex forms incorporating various field types, such as sliders and matrices. To simplify the creation process, especially for forms without complex logic, the tool offers different base templates. Furthermore, the form builder includes a robust validation engine, a theme engine, and a logic engine, providing extensive flexibility in controlling the look, feel, and flow of each form.

1.2 Scope

The tool will empower non-technical users to design, deploy, and manage a wide array of digital forms ranging from basic feedback surveys to sophisticated multi-page applications and complex quizzes without writing a single line of code. It achieves this by providing a highly intuitive drag-and-drop interface complemented by powerful backend engines for logic, validation, theming, and workflow integration.

The primary benefits and objectives are:

- **Simplified Form Creation:** Enable admin staff, students and professors to rapidly create forms.
- **Pre-Validated Data:** Ensure high-quality data collection through a robust Field Validator subsystem enforcing rules and constraints.
- **Process Automation:** Facilitate seamless integration into other processes via the Workflow Engine, automating actions post-submission.
- **Consistent Theming:** Allow organizations to maintain visual consistency across all forms using the centralized Theme Subsystem.
- **Flexibility for Complexity:** Support both simple, effortless form creation and highly complex requirements, such as conditional logic, sophisticated data validation, and quiz functionality.

The core features included in this scope are:

Core Form Construction and Design

- **Form Builder Core:** A responsive, drag-and-drop interface supporting the construction of simple and multi-page forms.
- **Component and Template Management:** Support for a rich library of standard and custom form components. This includes the ability to create, save, and reuse templates for entire forms, pages, and individual components, along with support for future AI-generated component suggestions.
- **Styling and Theming:** Implementation of a comprehensive Theme Subsystem allowing for detailed visual customization at the global, page, and component levels. It includes features for importing and exporting themes.

Advanced Functionality and Control

- **Logic and Flow Control (Logic Engine):** Tools for defining complex conditional logic, including component visibility based on prior input, dynamic component ordering, and basic data manipulation rules within the form.
- **Validation (Field Validator):** A subsystem for defining and enforcing data integrity rules, supporting standard validation types, regular expressions (regex), and custom constraints on field input.

Data Handling and Process Integration

- **Submission and Data Handling:** Secure processing of form submissions, reliable data storage, and a management interface allowing users to filter, view, and export submitted data.
- **Workflow Integration (Workflow Engine):** Functionality to define and automate processes (e.g., sending emails, initiating approvals) that are triggered immediately upon form submission.
- **Access Control (RBAC):** Implementation of Role-Based Access Control to govern permissions related to form creation, editing, filling (submission), and viewing submitted data.

Deployment and Distribution

- **Preview-Publish:** Mechanisms enabling users to preview the form's live behavior (including logic) before deployment. Support for publishing the form as a standalone webpage, generating embed codes, and integrating the form into other DISHA modules.

1.3 Definitions Acronyms And Abbreviations

Term	Definition
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
Form-Filler	The end-user who fills out and submits a published form.
Owner/Editor	A user responsible for creating and maintaining the form structure and logic.
Component	A fundamental building block of a form (e.g., text input, checkbox).
Field Validator	The subsystem enforcing data constraints and integrity rules on form components.
Theme Subsystem	The subsystem manages the visual presentation and styling of the form.
Logic Engine	The subsystem handles conditional flow, visibility, and ordering within the form.
Workflow Engine	The subsystem manages automated post-submission business processes.

1.4 Development Approach (Agile Context)

The system will be developed using an Agile model:

- Iterative sprint cycles
- Incremental feature delivery
- Continuous feedback from stakeholders
- Each sprint delivers testable functionality
- Requirements may evolve across iterations

Baseline functional requirements defined here are subject to refinement through sprint backlog grooming.

2. Overall Description

2.1 Product Perspective

The system is a web-based application consisting of:

- Form Builder Interface
- Schema Generator
- Form Renderer
- Validation Engine
- Logic Engine
- Workflow Engine
- Theme Subsystem
- Access Control Module
- Submission Manager
- AI Component Creator

The system internally represents every form as a structured JSON schema.

2.2 Product Functions

The system shall:

- Allow drag-and-drop form construction
- Support multi-page forms
- Support reusable custom components
- Provide field validation rules
- Provide conditional logic between components
- Support configurable workflows
- Provide theme customization
- Manage form versioning
- Support preview and publishing
- Store submissions
- Export submission data (CSV/JSON)
- Provide AI-assisted form creation
- Track time spent per question (analytics)

2.3 User Classes and Characteristics

Admin

- Create/edit forms
- Publish forms
- Configure workflows
- Manage roles

Editor

- Modify draft forms
- Suggest changes

Form-Filler

- View published forms
- Submit responses

Reviewer

- Approve/reject submissions based on workflow

2.4 Constraints

- Must support responsive design
- Must ensure secure submission storage
- Must allow JSON export for interoperability
- Must support iterative deployment (AGILE)

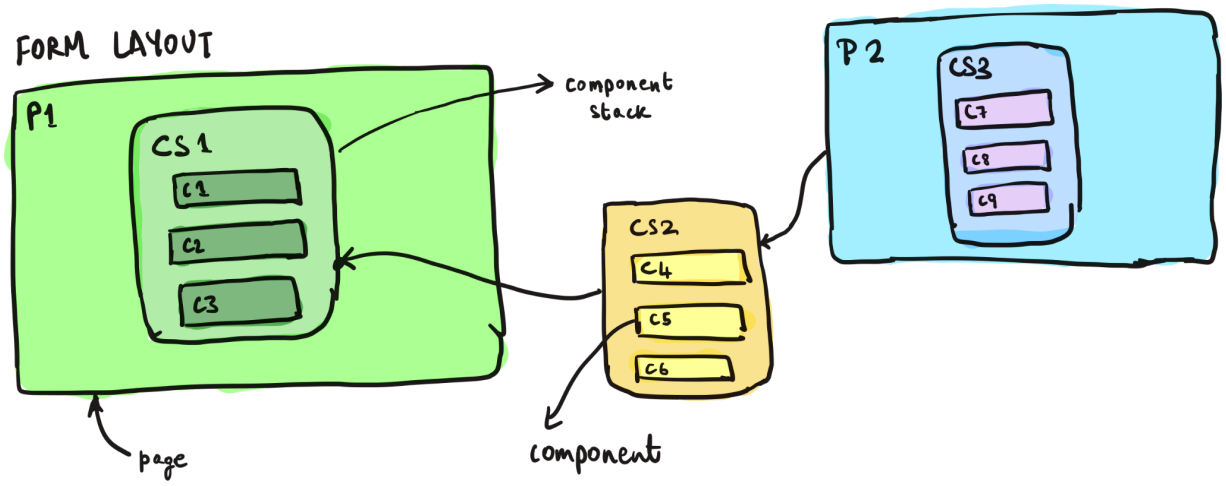
3. Functional Requirements

3.1 Form Builder

FR-1: Canvas Initialization
The system shall provide an empty canvas upon creation of a new form.

FR-2: Drag-and-Drop Components
The system shall allow users to drag and drop components into pages.

FR-3: Multi-Page Forms
The system shall allow multiple pages within a single form.



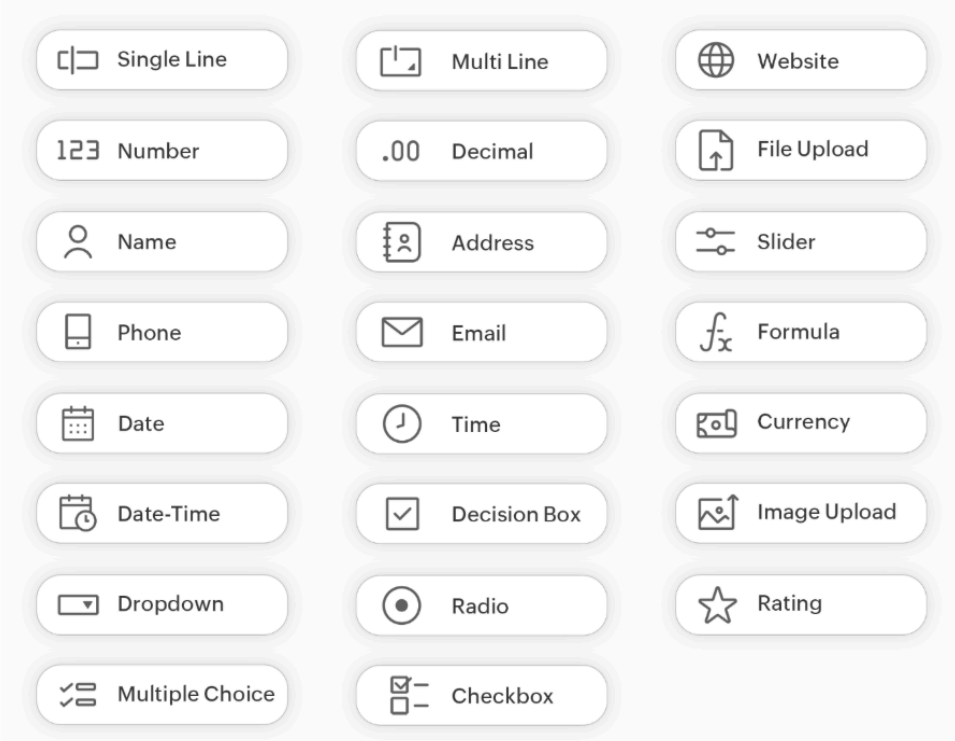
FR-4: Reordering
The system shall allow reordering of components within a page.

FR-5: Component Duplication
The system shall allow duplication of existing components.

FR-6: Component Library
The system shall provide the following built-in components:

- Single-line text
- Multi-line text
- Email
- Phone
- Number
- Decimal
- Date
- Time
- File upload
- Image upload
- Dropdown

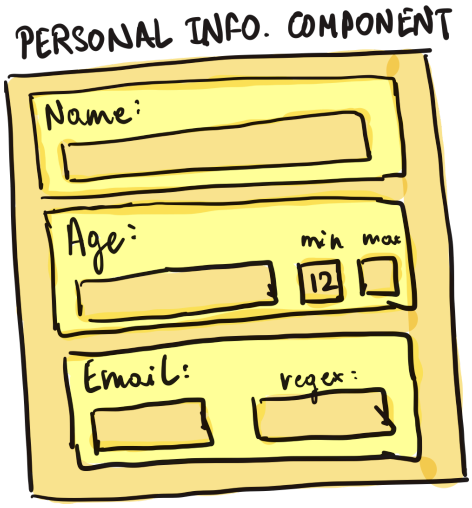
- Radio buttons
- Checkboxes
- Single-choice grid
- Multi-choice grid
- Matrix/table input
- Rating scale
- Linear scale
- Slider
- Address block
- Name block
- URL input
- Color picker
- Section divider
- Signature input
- CAPTCHA
- Page break
- Payment input (future extension)



3.2 Custom Components

FR-7: Composite Components
The system shall allow grouping multiple fields into reusable custom components.

FR-8: Template Storage
The system shall allow saving custom components into a reusable library.



3.3 Field Validation Engine

FR-9: Mandatory Fields
The system shall allow marking fields as required.

FR-10: Regex Validation
The system shall allow validation using regular expressions.

FR-11: Length Constraints
The system shall allow minimum and maximum length validation.

FR-12: Numeric Range
The system shall allow minimum and maximum numeric limits.

FR-13: File Validation
The system shall allow:

- File type restriction
- Maximum file size restriction

FR-14: Custom Error Messages
The system shall allow custom validation error messages.

FR-15: Cross-Field Validation
The system shall allow validation based on values of other fields.

3.4 Logic Engine

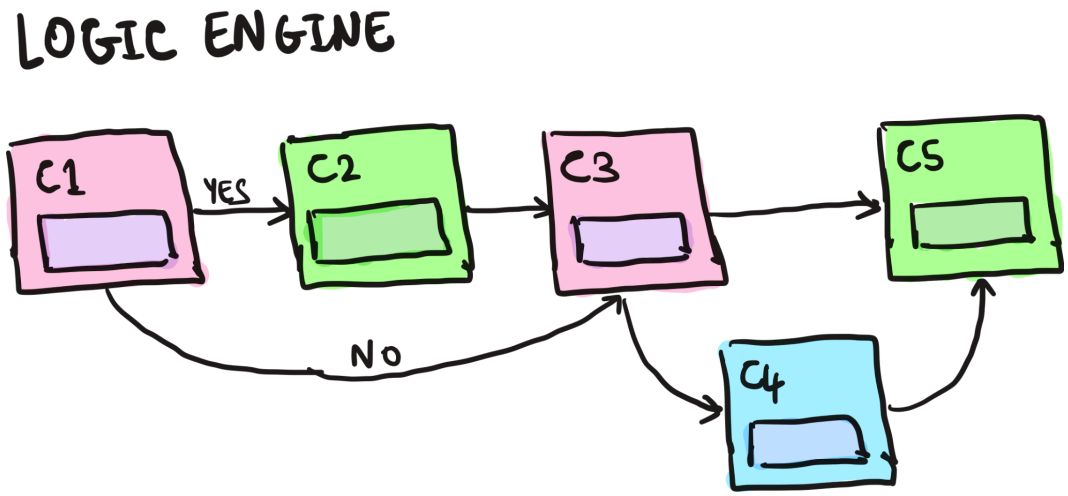
FR-16: Conditional Visibility
The system shall allow showing or hiding components based on conditions.

FR-17: Conditional Navigation
The system shall allow skipping pages based on answers.

FR-18: Option Shuffling
The system shall allow randomizing option order.

FR-19: Calculated Fields
The system shall allow computed fields based on formulas.

FR-20: Topological Execution
The system shall internally maintain dependency order of components.



3.5 Theme Subsystem

FR-21: Global Theme
The system shall allow setting global themes.

FR-22: Page-Level Theme
The system shall allow per-page customization.

FR-23: Component-Level Theme
The system shall allow component-specific styling.

FR-24: Custom Theme Export/Import
The system shall allow exporting and importing themes.

FR-25: Dark/Light Mode
The system shall support theme modes.

3.6 Workflow Engine

FR-21: State Definition
The system shall allow defining workflow states.

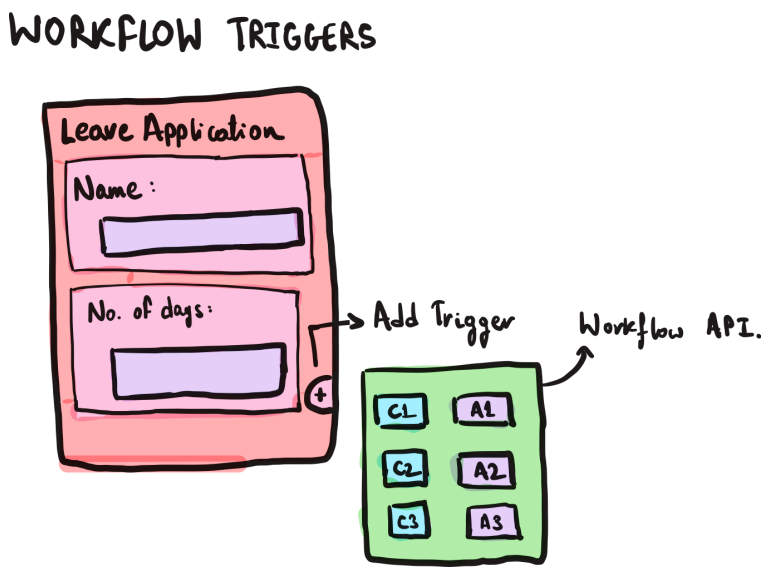
FR-22: Transitions
The system shall allow transitions between states.

FR-23: Conditional Transitions
Transitions shall support conditions.

Example:
If `leave_days <= 3` → auto approve
Else → forward to manager

FR-24: Role Assignment
Transitions shall be restricted to assigned roles.

FR-25: Audit Trail
All workflow transitions shall be logged.



3.7 Versioning

FR-31: Automatic Version Creation
Each save shall create a version snapshot.

FR-32: Version Comparison
The system shall allow comparing versions.

FR-33: Rollback
The system shall allow restoring previous versions.

3.8 Preview And Publish

FR-34: Real-Time Preview
The system shall provide live preview during editing.

FR-35: Device Simulation
The system shall simulate desktop, tablet, and mobile views.

FR-36: Draft Saving
The system shall allow saving drafts.

FR-37: Publishing
The system shall allow publishing only validated versions.

3.9 Submissions

FR-38: Store Submission
The system shall store responses securely.

FR-39: Update Draft Submission
The system shall allow saving partially completed forms.

FR-40: Export Data
The system shall allow exporting submissions to CSV and JSON.

FR-41: Time Analytics
The system shall record time spent per question.

3.9 Access Control (RBAC)

FR-42: Role Creation
The system shall allow defining roles.

FR-43: Permission Assignment
Permissions shall be assignable per role.

FR-44: Enforcement
All API operations shall validate role permissions.

3.10 AI-Assisted Form Generation

FR-45: Text-to-Form
The system shall allow generating a form from a natural language prompt.

FR-46: Validation Suggestions
The AI shall suggest appropriate validations.

FR-47: Workflow Suggestions
The AI shall suggest workflow states and transitions.

FR-48: Component Generator
The AI shall generate composite components.

FR-49: Schema Generation
The AI shall output valid JSON schema compatible with the system.

4. JSON Schema Specification

Every form shall be stored as structured JSON.

4.1 Schema Requirements

The schema shall include:

- Form metadata
- Pages
- Components
- Validation rules
- Logic conditions
- Workflow configuration
- Theme configuration
- Version identifier

4.2 Sample JSON Structure

```
{
  "formId": "leave_request",
  "version": 3,
  "pages": [
    {
      "pageId": "p1",
      "components": [
          {
             "type": "number",
             "id": "leave_days",
             "label": "Number of Leave Days",
             "validation": {
             "required": true,
             "min": 1,
             "max": 30
          },
          ...
        },
        ...
      ]
    }
  ],
  "workflow": {
    "states": ["submitted", "manager_review", "approved"],
    "transitions": [
      {
        "from": "submitted",
        "to": "approved",
        "condition": "leave_days <= 3"
      },
       ...
```

```
    ]
  }
}
```

4.2 JSON Requirements

- Must be versioned
- Must be AI-editable
- Must validate against schema definition

5. Non-Functional Requirements

5.1 Performance

- Page load < 3 seconds
- Support 100+ field forms

5.2 Security

- Input sanitization
- Secure file uploads
- Role-based access enforcement

5.3 Reliability

- Version rollback must not corrupt data
- Workflow transitions must be atomic

5.4 Usability

- Designed for non-technical users
- Minimal configuration required

6. API Interface

6.1 Overview

The system shall expose a REST-based API interface to support:

- Form creation and management
- JSON schema retrieval
- Submission handling
- Workflow transitions
- Access control enforcement
- AI-assisted generation
- Version management
-

The API shall be stateless and follow standard HTTP semantics.
All APIs shall accept and return JSON.

6.2 Authentication & Authorization

Authentication
The system shall use token-based authentication.

- Upon login, the system shall issue an access token.
- The token shall be included in the Authorization header.
- Tokens shall expire after a configurable duration.
- Token refresh mechanism may be supported.

Authorization
Each request shall:

- Extract user role from token.
- Validate role permissions.
- Enforce object-level access control (e.g., form ownership).

If unauthorized:

- Return HTTP 403.

6.2 Core API Categories

A. Form Management APIs

1. Create Form
   `POST /api/v1/forms`

   Request:
   - Metadata
   - Initial schema (optional)

   Response:

- formId
- versionId
- status (draft)

2. Get Form Schema
   `GET /api/v1/forms/{formId}`

   Returns:
   - Latest draft or published version
   - Full JSON schema
   - Workflow configuration
   - Theme configuration

3. Update Draft
   `PUT /api/v1/forms/{formId}/draft`

   Request:
   - Updated JSON schema

   System Behavior:
   - Validate schema structure
   - Create new version snapshot
   - Return versionId

4. Publish Form
   `POST /api/v1/forms/{formId}/publish`

   System Behavior:
   - Validate:
     - No missing required components
     - Workflow integrity
     - Validation rules correctness
   - Lock version
   - Mark as published
5. Get Versions
   `GET /api/v1/forms/{formId}/versions`

   Returns:
   - List of version metadata
   - Timestamps
   - Author
   - Status

6. Rollback Version
   `POST /api/v1/forms/{formId}/rollback/{versionId}`

Creates a new version based on selected snapshot.

B. Submission APIs

1. Submit Form
   `POST /api/v1/forms/{formId}/submit`

   Request:
   - Submission JSON
   - System Behavior:
   - Validate fields
   - Apply logic engine
   - Evaluate workflow transitions
   - Store submission
   - Trigger notifications

   Response:
   - submissionId
   - workflowState

2. Save Draft Submission
   `POST /api/v1/forms/{formId}/draft-submission`

   Stores partial submission.

3. Get Submission
   `GET /api/v1/submissions/{submissionId}`

   Access:
   Only authorized users

4. Export Submissions
   `GET /api/v1/forms/{formId}/export?format=csv`

   Supported formats:
   - CSV
   - JSON

C. Workflow APIs

1. Define Workflow
   `PUT /api/v1/forms/{formId}/workflow`

   Request:
   - States
   - Transitions

- Conditions
- Role mapping

2. Transition Submission
   `POST /api/v1/submissions/{submissionId}/transition`

   Request:
   - targetState

   System Behavior:
   - Validate current state
   - Validate role
   - Check transition condition
   - Update state
   - Log audit record

7. Verification & Validation

7.1 Overview

Verification and Validation ensure that:

- Verification: The system is built correctly (meets specified requirements).
- Validation: The system solves the intended user problem.

Testing shall be iterative and integrated into each Agile sprint.

- The testing strategy shall include:
- Unit Testing
- Integration Testing
- API Testing
- System Testing
- Regression Testing
- Performance Testing
- Security Testing
- Usability Testing
- User Acceptance Testing (UAT)

7.2 Requirement Verification Method

Each requirement shall be verified using one or more of the following methods:

Method	**Description**
Inspection	Manual review of artifacts
Unit Test	Automated code-level tests
Integration Test	Multi-module interaction testing
System Test	End-to-end functional validation
Demonstration	Live sprint demo validation
User Testing	Testing with real end-users

8. Testing Levels

8.1 Unit Testing

Unit testing shall validate individual modules:

- Field Validation Engine
- Logic Engine
- Workflow State Machine
- JSON Schema Validator
- Access Control Middleware
- API Handlers

Requirements:

- Each functional module shall have ≥ 80% unit test coverage.
- Edge cases shall be tested (empty fields, invalid JSON, circular logic, etc.).

Example:

- Test that numeric validation rejects out-of-range values.
- Test that workflow rejects invalid transitions.
- Test that circular dependencies in logic are detected.

8.2 Integration Testing

Integration testing shall validate interactions between:

- Form Builder → JSON Generator
- JSON Schema → Renderer
- Submission → Workflow Engine
- Workflow Engine → Notification System
- AI Generator → Schema Validator

Test cases shall verify:

- Correct propagation of validation rules
- Correct execution order of logic dependencies
- Correct role-based access enforcement

8.3 API Testing

All APIs shall be tested for:

- Correct status codes
- Correct JSON responses
- Role-based access enforcement
- Invalid input handling
- Version consistency

Tests shall verify:

- Unauthorized users cannot publish forms.
- Invalid schemas return proper validation errors.
- Submissions trigger correct workflow transitions.
- Automated API testing shall be integrated into CI/CD.

8.4 System Testing (End-to-End)

System testing shall simulate real-world scenarios.

Example Scenarios:

- Admin creates form → publishes → user submits → workflow transitions → reviewer approves.
- Admin edits form → version created → rollback → verify integrity.
- AI generates form → schema validated → form published → submission processed.

8.4 JSON Schema Verification

The system shall verify:

- Structural correctness (required fields present).
- Logical correctness (no circular references).
- Workflow integrity (all states reachable).
- Backward compatibility across versions.
- AI-generated schema normalization.

A formal schema definition (JSON Schema standard) shall be maintained.
All schemas must pass automated schema validation before being stored.

8.5 Workflow Engine Testing

The workflow engine shall be tested using:

- State transition coverage testing
- Invalid transition rejection testing
- Role enforcement testing
- Condition evaluation testing
- Audit log verification
- Formal verification criteria:
- Every defined state must be reachable.
- No transition shall exist without defined roles.
- No undefined states shall be referenced.

Edge cases:

- Concurrent transition attempts
- Condition evaluating to null
- Manual override scenarios

8.6 Validation Engine Testing

Validation testing shall include:

- Required field enforcement
- Regex validation accuracy
- Numeric range boundaries
- File size/type rejection
- Cross-field dependencies
- Error message correctness

Negative test cases shall be prioritized.

8.7 Logic Engine Testing

The system shall test:

- Conditional visibility
- Page skipping logic
- Option randomization
- Calculated field accuracy
- Dependency graph topological ordering
- The logic engine must detect:
- Circular dependencies
- Unreachable components
- Conflicting rules

8.8 Performance Testing

Performance testing shall verify:

- Form load time < 3 seconds (under normal load)
- Submission processing under concurrent users
- Large form handling (100+ fields)
- Export performance for large datasets
- Stress testing shall simulate:
- Concurrent submissions
- Simultaneous workflow transitions
- Large export requests

8.9 Security Testing

- Security testing shall include:
- Input sanitization tests (XSS prevention).
- API authentication enforcement.
- Role escalation prevention.
- File upload validation.
- JSON injection prevention.
- Token expiration handling.
- Penetration-style testing shall be conducted before final release.

8.10 Usability Testing

Usability validation shall include:

- First-time user testing
- Task completion time measurement
- Error recovery testing
- Accessibility testing (contrast, responsiveness)

Test scenarios:

- Create a leave request form in under 10 minutes.
- Add validation without documentation.
- Configure a simple workflow without training.

Usability metrics:

- Task success rate
- Average form creation time
- Error frequency

8.11 User Acceptance Testing (UAT)

Before production release:

- Feedback shall be collected.
- Critical usability or functional issues shall be resolved.

Acceptance Criteria:

- All critical and high-priority defects resolved.
- No blocking workflow errors.
- All core APIs are functioning.

8.12 Regression Testing

Each sprint shall include regression testing to ensure:

- Existing features remain stable.
- Version rollback remains functional.
- AI integration does not corrupt schemas.
- Automated regression tests shall run after every major update.

8.13 Continuous Testing in Agile

Each sprint shall include:

- Sprint-level unit tests
- Integration tests for new modules
- Demo verification against requirements
- Bug backlog refinement
- Regression testing of critical modules

Definition of Done (DoD):

- A feature is complete only if:
- Functional tests pass
- Unit coverage threshold met
- API contract validated
- Security checks passed
- Reviewed by QA

9. Vital, Essential and Desirable Features

This section prioritizes system capabilities to support Agile planning, MVP definition, and incremental delivery.

9.1 Vital Features

These features are mandatory for the system to be considered operational.

- Core Form Functionality
- Drag-and-drop form builder
- Multi-page form support
- Standard component library
- JSON schema generation and storage
- Real-time preview
- Draft saving
- Publishing mechanism
- Data Integrity
- Required field validation
- Numeric range validation
- Regex validation
- File size/type validation
- Schema structural validation
- Submission Handling
- Secure submission storage
- Basic submission retrieval
- CSV export
- Workflow (Basic)
- Access Control
- API-level enforcement
- Versioning
- Rollback capability

Without these features, the system cannot deliver its primary objective.

9.2 Essential Features

These features significantly enhance usability and robustness.

- Advanced Logic
- Conditional visibility
- Conditional navigation (page skipping)
- Calculated fields
- Dependency resolution engine
- Theming
- Global theme configuration

- Page-level styling
- Dark/Light mode
- Preview & Testing
- Device simulation (mobile/tablet/desktop)
- Logic simulation in preview mode
- Workflow Enhancements
- Conditional transitions
- Workflow visualization
- Notification triggers (email)
- API Enhancements
- API versioning
- Audit logging
- Error standardization
- Pagination for large datasets
- Security Enhancements
- Token expiration handling
- Object-level access control
- Input sanitization (XSS prevention)

These features make the system enterprise-ready.

9.3 Desirable Features

These features provide strategic advantage and competitive differentiation

- AI Capabilities
- Full text-to-form generation
- AI-suggested validation rules
- AI-suggested workflow structures
- AI-generated reusable components
- Advanced Analytics
- Time spent per question
- Completion drop-off analysis
- Submission trend dashboards
- AI-powered insight summaries
- Theme Marketplace
- Import/export theme packages
- Template Library Expansion (Quiz, Application,Feedback templates)
- Webhook integrations
- External API integrations

Desirable features shall be implemented after MVP stabilization.

10. Minimum Viable Product

10.1 MVP Objective

The MVP shall deliver a fully usable no-code form builder that supports basic workflows and secure submissions, without advanced AI or analytics.

The MVP must demonstrate:

- Usability for non-technical users
- Data integrity
- Secure submission processing
- Basic workflow automation

10.2 MVP Feature Set

- Core Builder
- Drag-and-drop interface
- Multi-page forms
- Standard components
- Component reordering
- Draft saving
- Validation
- Required fields
- Numeric range
- Regex
- File validation
- Logic
- Conditional visibility
- Basic calculated fields
- Workflow
- State creation
- Transition definition
- Role-based transition control
- Access Control
- Admin and Form-Filler roles
- Permission enforcement
- Submissions
- Store submission
- Retrieve submission
- Export CSV
- Versioning
- Automatic version creation
- Rollback support
- Preview & Publish
- Publish form

10.3 Features Excluded from MVP

The following shall be excluded from MVP:

- AI-based form generation
- Advanced analytics dashboard
- Marketplace of themes
- Advanced AI insights

These shall be part of Phase 2 development.

11. Technology Stack

This section specifies the primary tools and technologies that shall be used to develop, test, and deploy the Dynamic Form Builder and Workflow System.

11.1 Frontend

The frontend shall be developed using:

- React.js – UI framework
- TypeScript – Strongly typed development
- Tailwind CSS – Styling and responsive design
- React Hook Form – Form state management
- Zod – Schema validation

11.2 Backend

The backend shall be developed using:

- Node.js – Runtime environment
- Express.js – REST API framework
- JWT – Authentication
- bcrypt – Password hashing

11.3 Database

The system shall use:

- MongoDB – Primary database (JSON document storage)
- Mongoose – Schema modeling and validation

11.4 AI Integration

For AI-based form generation:

- External LLM API or a locally trained model
- AJV – JSON schema validation

11.5 Testing

- Jest – Unit testing
- Postman – API testing

12. Agile Sprint Structure

Each sprint duration: 2 weeks

Each sprint includes:

- Planning
- Development
- Testing
- Demo
- Retrospective

Sprint 2: API Foundation & Core Schema Engine

Goal: Establish backend contract.

Deliverables:

- Authentication module
- Form CRUD APIs
- JSON schema storage
- Versioning mechanism
- Basic submission endpoint (no workflow yet)
- Schema validation engine

Outcome:
Working API that can create and retrieve forms.

Sprint 3: Validation Engine & Logic Engine

Goal: Make forms dynamic.
Deliverables:

- Field validation engine
- Conditional logic execution engine
- Cross-field dependency resolution
- Calculated fields
- JSON logical validation
- Preview API endpoint

Outcome:
Forms behave dynamically.

Sprint 4: Workflow Engine & RBAC

Goal: Introduce process control.

Deliverables:

- State machine engine
- Transition validation
- Role-based access enforcement
- Workflow assignment API
- Audit logging

Outcome:
Submission transitions are controlled and secure.

Sprint 5: AI Integration & JSON Intelligence

Goal: Intelligent schema generation.

Deliverables:

- AI prompt → JSON generator
- AI validation suggestions
- AI workflow suggestions
- JSON normalization pipeline
- Safety validation of AI output

Outcome:
Users can generate forms from text.

Sprint 5: Performance & Analytics

Goal: Production readiness.

Deliverables:

- Export system (CSV/JSON)
- Time-per-question analytics
- API pagination
- Error standardization
- Performance optimization
- Security hardening
- Documentation finalization

Outcome:
The system is stable and scalable.